

Fun with LaTeX

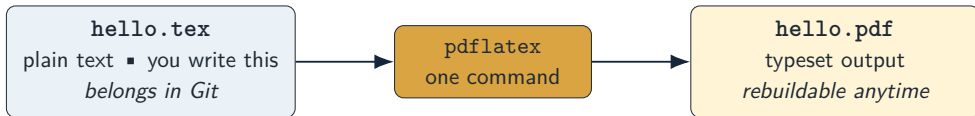
Your First Document That Builds Itself

The whole idea in one sentence

`hello.tex` is source code. `hello.pdf` is a build artifact. Git tracks the source; one command rebuilds the PDF.

Companion slides for the recorded walkthrough – see the Fun with LaTeX page on Canvas for the video and the student guide.

Source code vs. build artifact



The one question to keep asking

Which file could I always regenerate, and which one is the thing I actually authored?
Track the source. The artifact is a result.

Predict before you run anything

1. If `hello.tex` is source and `hello.pdf` is built from it, which file belongs in Git?
2. `pdflatex.exe` is on disk. Does the file existing prove the tool *works*?
3. A step deletes four temp files; only two exist. Should it succeed or fail?

Write your answers down

Every one of these came up as a real snag in the recording. We return to them at the end.

Open PowerShell – literally the first step

If you have never opened it

Press the **Windows key**, type powershell, press **Enter**. A dark window with a blinking prompt appears. That prompt is where every command goes.

Get to a working folder

```
cd ~  
mkdir git -Force ; cd git  
git clone https://github.com/jeremy-evert/computing_commons.git  
cd computing_commons\fun_with_LaTeX
```

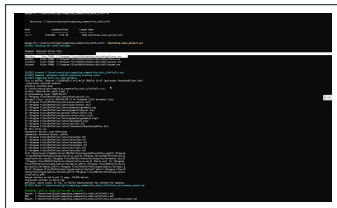
Run Get-Location and ls. If you see bootstrap_latex_project.ps1, you are in the right place.

Run the bootstrap script once

```
.\bootstrap_latex_project.ps1
```

It does six things:

1. checks the LaTeX toolchain
2. installs MiKTeX *only if* pdflatex is missing
3. writes hello.tex (won't overwrite yours)
4. adds build junk to .gitignore
5. compiles with pdflatex
6. writes latex_environment_report.md



What the toolchain check proves – and doesn't

[illegible]

Read the table

pdflatex FOUND means the build can run. The report file it writes – computer name, tool paths, Compiler exit code: 0 – is your evidence the PDF built *on your machine*.

The warning that is not a failure

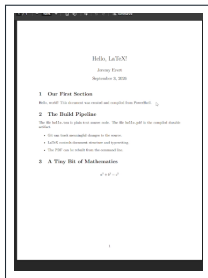
You will see this in red-ish text

```
pdflatex: major issue: So far, no MiKTeX administrator has checked  
for updates.
```

It did not stop the build

The very next lines say [SUCCESS] LaTeX is ready and the PDF was built. A warning is not a failed build. Open MiKTeX Console and check for updates when convenient – then move on.

Open the artifact you just built



Verify, don't assume

One page: title, your name, today's date, three sections, and $a^2 + b^2 = c^2$ as real math.

The habit

A clean exit code is not proof. Open the PDF and check it is what you meant to produce.

Edit, rebuild, see the change

The entire skill is this loop

```
code .\hello.tex          # change one true sentence, save
```

```
pdflatex -interaction=nonstopmode -halt-on-error \  
-file-line-error hello.tex
```

```
Start-Process .\hello.pdf
```

Edit source → run one command → inspect the artifact. Everything else is convenience on top of this.

The Makefile detour, snag 1: latexmk needs Perl

latexmk.exe was on disk, so it looked usable

MiKTeX could not find the script engine 'perl'
which is required to execute 'latexmk'.

Prediction #2, answered

Finding an executable does not prove its dependencies are satisfied. pdflatex alone already builds this document – so call it directly and depend on nothing else.

The Makefile detour, snag 2: cleanup that fails on nothing

Deleting temp files that were already gone

```
make: *** [Makefile:32: clean] Error 1
```

The cleanup command returned non-zero because some files did not exist. `make` correctly called that a failed step – and `rebuild` depended on `clean`, so `pdflatex` never ran.

Prediction #3, answered

“Delete these if they exist” should *succeed* when they are already gone.

The fix was subtraction

The working Makefile – four functional lines

```
.PHONY: all
all:
    pdflatex -interaction=nonstopmode -halt-on-error \
        -file-line-error hello.tex
    powershell.exe -NoProfile -Command \
        "Start-Process (Resolve-Path 'hello.pdf')"
```

No `latexmk`, no Perl, no cleanup target, no dependency graph. Recipe lines start with a real **tab**.

The rule

Start with the command that works. Automate *that*. Add complexity only when the project proves it needs it.

Keep the receipt (Git)

```
git status
git diff --cached
git diff
git add .
git commit -m "Initial commit"
git push
```

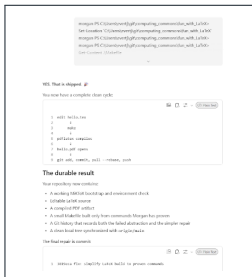
Commit the history, snags and all

status → add → diff -cached → commit → pull -rebase → push. The history records both the failed abstraction and the simpler repair – not something to hide.

The finish line

You are done when you can say

I turned a text file into a PDF on my own machine with one command. I have the environment report as evidence it built here. I know which file is source and which is the artifact, and both are under Git. I hit a build failure, read the real error, removed what did not earn its place, and got to a working make.



Where to go from here

- ▶ Full walkthrough video and the step-by-step student guide: the **Fun with LaTeX** page in Computing Commons on Canvas.
- ▶ Stuck? The student guide has an “If something goes wrong” table with the exact snags from this recording.
- ▶ Next: change the document – add a section, a table, a figure – then make again and watch the artifact follow your source.

The station question

Can another human understand and inspect what I did? A tracked .tex source plus a rebuildable .pdf is one honest answer.